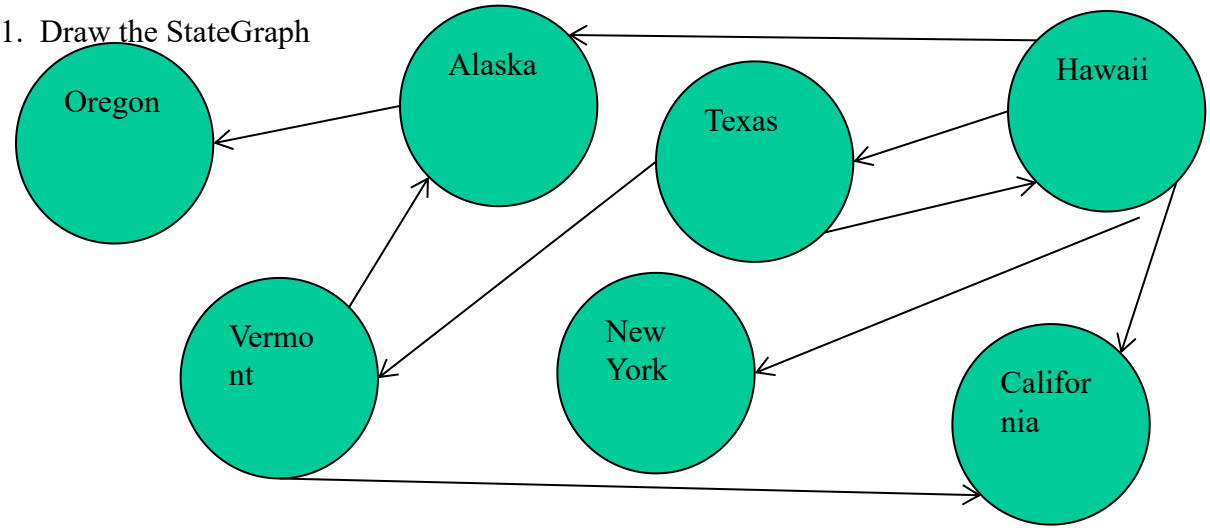
CMSC204
Kartchner

V(StateGraph) = {Oregon, Alaska, Texas, Hawaii, Vermont, NewYork, California}
E(StateGraph) = {(Alaska, Oregon), (Hawaii, Alaska), (Hawaii, Texas), (Texas, Hawaii), (Hawaii, California), (Hawaii, New York), (Texas, Vermont), (Vermont, California), (Vermont, Alaska)}

1. Draw the StateGraph



1. Describe the graph pictured above, using the formal graph notation.

V(StateGraph) = {Oregon, Alaska, Texas, Hawaii, Vermont, New York, Califoria}

E(StateGraph) = {(Alaska, Oregon), (Hawaii, Alaska), (Hawaii, Texas), (Texas, Hawaii), (Hawaii, California), (Hawaii, New York), (Texas, Vermont), (Vermont, California), (Vermont, Alaska)}

2. a. Is there a path from Oregon to any other state in the graph?
   a) No

   b. Is there a path from Hawaii to every other state in the graph?
      a) Yes, it doesn't go to Oregon or Vermont diectly but can get there through Texas and Alaska.

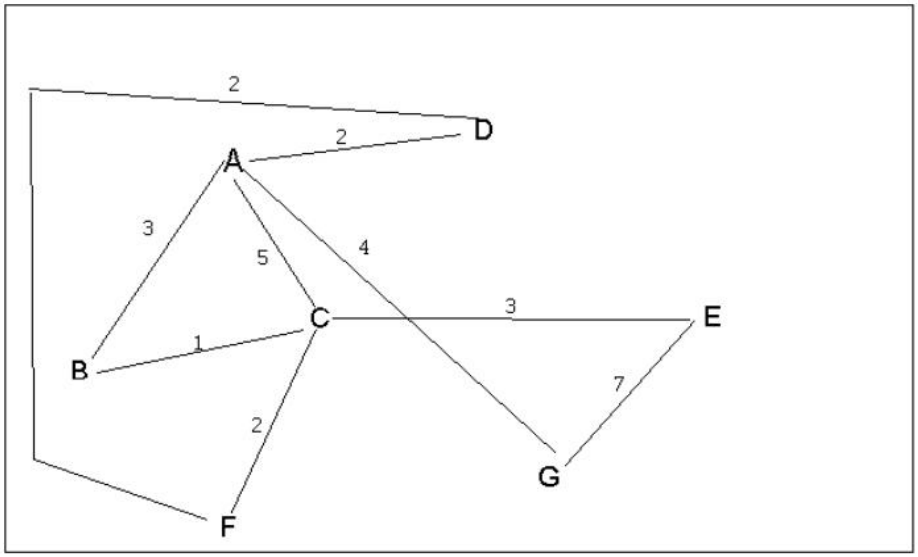
   c. From which state(s) in the graph is there a path to Hawaii?
      a) Only from Texas, and Hawaii if counting itself.

3. a. Show the adjacency matrix that would describe the edges in the graph.
Store the vertices in alphabetical order

States	AK	CA	HI	NY	OR	TX	VT
Alaska	0	0	0	0	1	0	0
California	0	0	0	0	0	0	0
Hawaii	1	1	0	1	0	1	0
New York	0	0	0	0	0	0	0
Oregon	0	0	0	0	0	0	0
Texas	0	0	1	0	0	0	1
Vermont	1	1	0	0	0	0	0

3. b. Show the adjacency lists
that would describe the edges in the graph

Alaska: Oregon
California: N/A
Hawaii: Alaska, California, New York, Texas
New York: N/A
Oregon: N/A
Texas: Hawaii, Vermont
Vermont: Alaska, California



4 a. Which of the following lists the graph nodes in depth first order beginning with E?

A) E, G, F, C, D, B, A

B) G, A, E, C, B, F, D

C) E, G, A, D, F, C, B

D) E, C, F, B, A, D, G

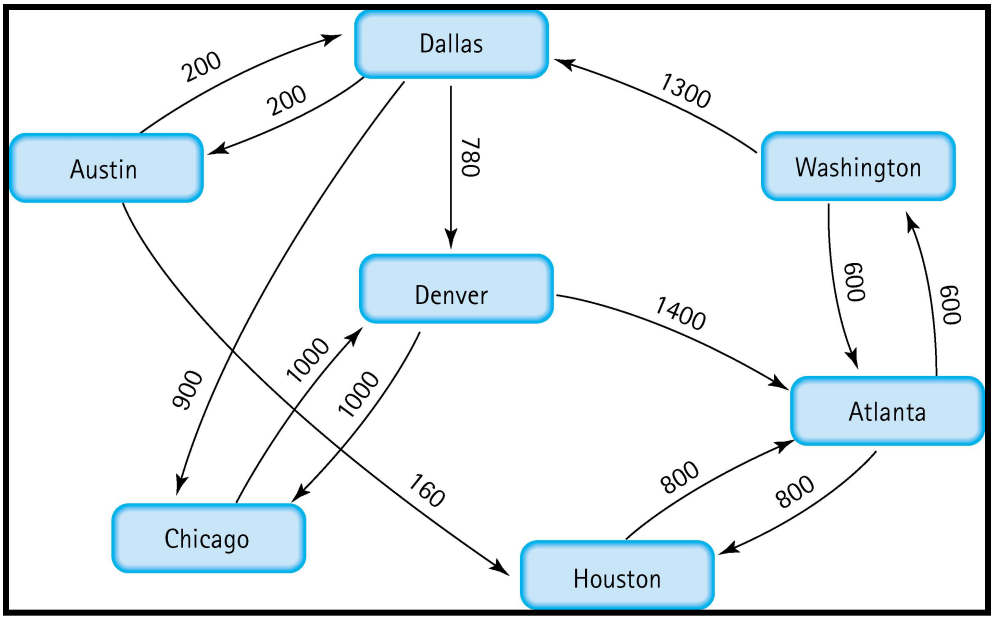
4 b. Which of the following lists the graph nodes in breadth first order beginning at F?

A) F, C, D, A, B, E, G

B) F, D, C, A, B, C, G

C) F, C, D, B, G, A, E

D) a, b, and c are all breadth first traversals



5. Find the shortest <u>distance</u> from Atlanta to every other city

Washington: (Atlanta, Washington) --> 600

Houston: (Atlanta, Houston) --> 800

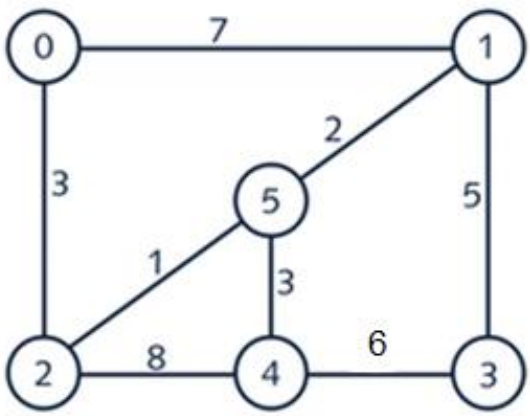
Dallas: {(Atlanta, Washington) 600, (Washington, Dallas) 1300 } --> 1900

Austin: {(Atlanta, Washington) 600, (Washington, Dallas) 1300, (Dallas, Austin) 200 } --> 2100

Denver: {(Atlanta, Washington) 600, (Washington, Dallas) 1300, (Dallas, Denver) 780 } --> 2680

Chicago: {(Atlanta, Washington) 600, (Washington, Dallas) 1300, (Dallas, Chicago) 900 } --> 2800

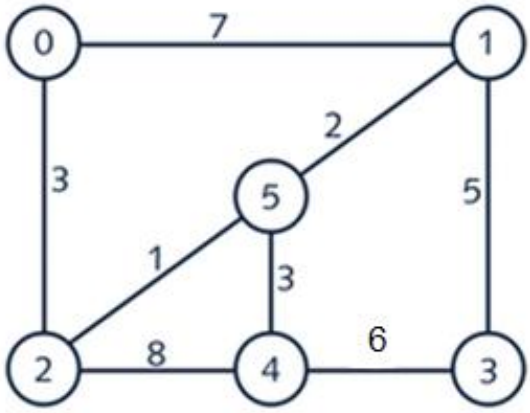
6. Find the minimal spanning tree using Prim's algorithm. Use 0 as the source vertex . Show the steps.



Step	Current Set of Nodes	Available Edges (connected to current nodes)	Choice (Cheapest Edge)	New Node Added
1	{0}	(0,1) weight 7, **(0,2) weight 3**	(0,2)	2
2	{0, 2}	(0,1) weight 7, **(2,5) weight 1**, (2,4) weight 8	(2,5)	5
3	{0, 2, 5}	(0,1) weight 7, (2,4) weight 8, **(5,1) weight 2**, (5,4) weight 3	(5,1)	1
4	{0, 2, 5, 1}	(2,4) weight 8, **(5,4) weight 3**, (1,3) weight 5	(5,4)	4
5	{0, 2, 5, 1, 4}	(1,3) weight 5, **(4,3) weight 6**	**(1,3)**	3

Final minimal spanning tree = {(0,2), (2,5), (5,1), (5,4), (1,3)}

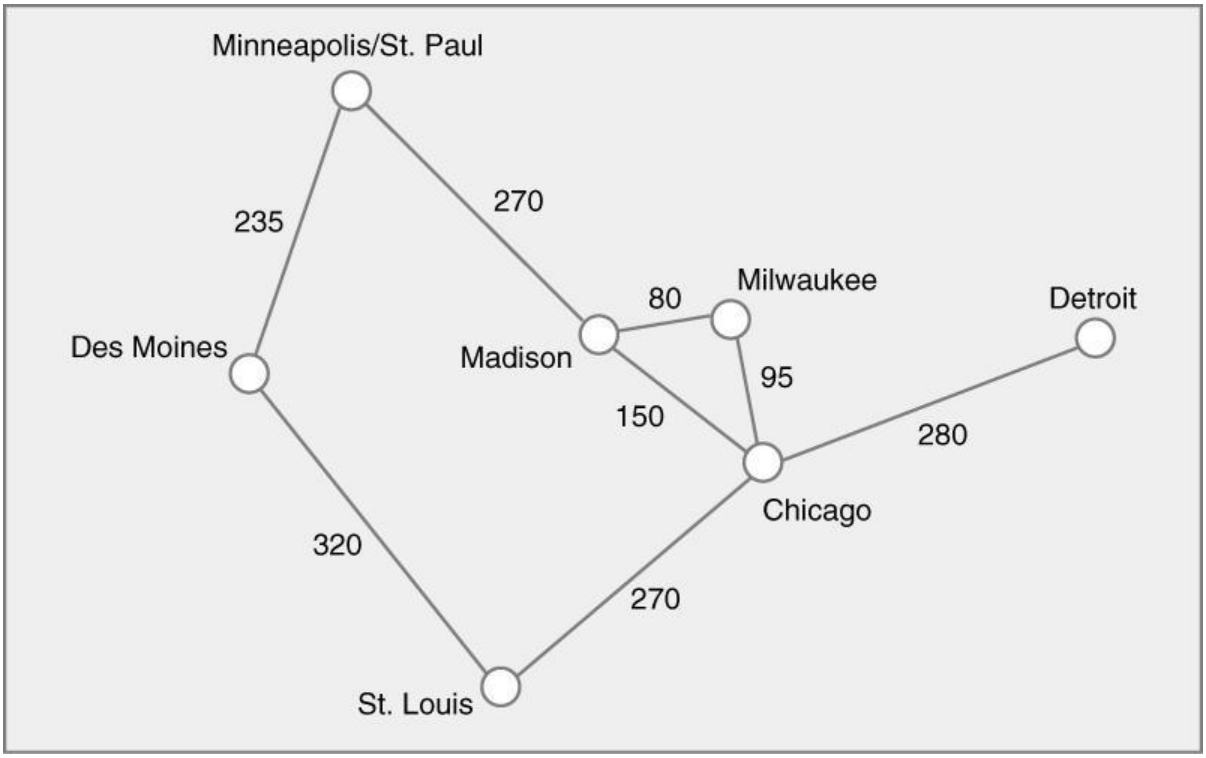
7. Find the minimal spanning tree using Kruskal's algorithm. Show the weights in order and the steps.



Step	Edge	Weight	Action	Reason
1	(2, 5)	1	**Accepted**	Smallest weight; does not form a cycle.
2	(1, 5)	2	**Accepted**	Next smallest; does not form a cycle.
3	(0, 2)	3	**Accepted**	Does not form a cycle.
4	(4, 5)	3	**Accepted**	Does not form a cycle.
5	(1, 3)	5	**Accepted**	Connects the final vertex (3) without a cycle.
6	(3, 4)	6	Rejected	Vertices 3 and 4 are already connected (via 3-1-5-4).
7	(0, 1)	7	Rejected	Would create a cycle (0-2-5-1-0).
8	(2, 4)	8	Rejected	Would create a cycle (2-5-4-2)

Final MST edges = {(2,5), (1,5), (0,2), (4,5), (1,3)}

8. Find the minimal spanning tree using the algorithm you prefer. Use Minneapolis/St. Paul as the source vertex

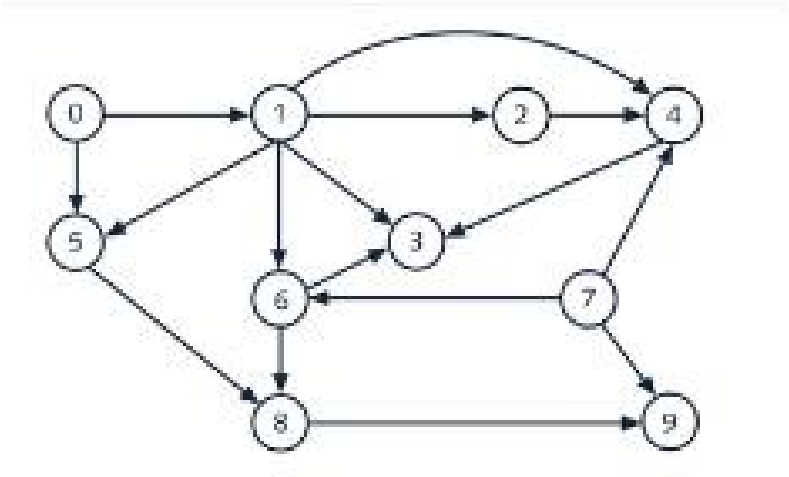


Step	Current Set of Nodes	Available Edges (connected to current nodes)	Choice (Cheapest Edge)	New Node Added
1	{Minneapolis/St. Paul}	**(Minneapolis/St. Paul, Des Moines) weight 235. (Minneapolis/St.Paul, Madison) weight 270.**	(Minneapolis/St.Paul, Des Moines)	**Des Moines**
2	{Minneapolis/St.Paul, Des Moines}	(Minneapolis/St.Paul, Madison) weight 270, (Des Moines, St.Louis) weight 320	(Minneapolis/St.Paul, Madison)	**Madison**
3	{Minneapolis/St.Paul, Des Moines, Madison}	(Des Moines, St.Louis) weight 320, (Madison, Milwaukee) weight 80, (Madison, Chicago) weight 150	(Madison, Milwaukee)	**Milwaukee**
4	{Minneapolis/St.Paul, Des Moines, Madison, Milwaukee}	(Des Moines, St.Louis) weight 320, (Madison, Chicago) weight 150, (Milwaukee, Chicago) weight 95	(Milwaukee, Chicago)	**Chicago**
5	{Minneapolis/St.Paul, Des Moines, Madison, Milwaukee, Chicago}	(Des Moines, St.Louis) weight 320, (Chicago, Detroit) weight 280, (Chicago, St.Louis) weight 270	**(Chicago, St.Louis)**	**St.Louis**
6	{Minneapolis/St.Paul, Des Moines, Madison, Milwaukee, Chicago, St.Louis}	(Chicago, Detroit) weight 280	**(Chicago, Detroit)**	**Detroit**

MST = {(Minneapolis/St.Paul, Des Moines), (Minneapolis, Madison), (Madiosn, Milwaukee), (Milwaukee, Chicago), (Chicago, St.Louis), (Chicago, Detroit)}

9. List the nodes of the graph in a breadth first topological ordering. Show the steps using arrays predCount, topologicalOrder and a queue

*FIFO



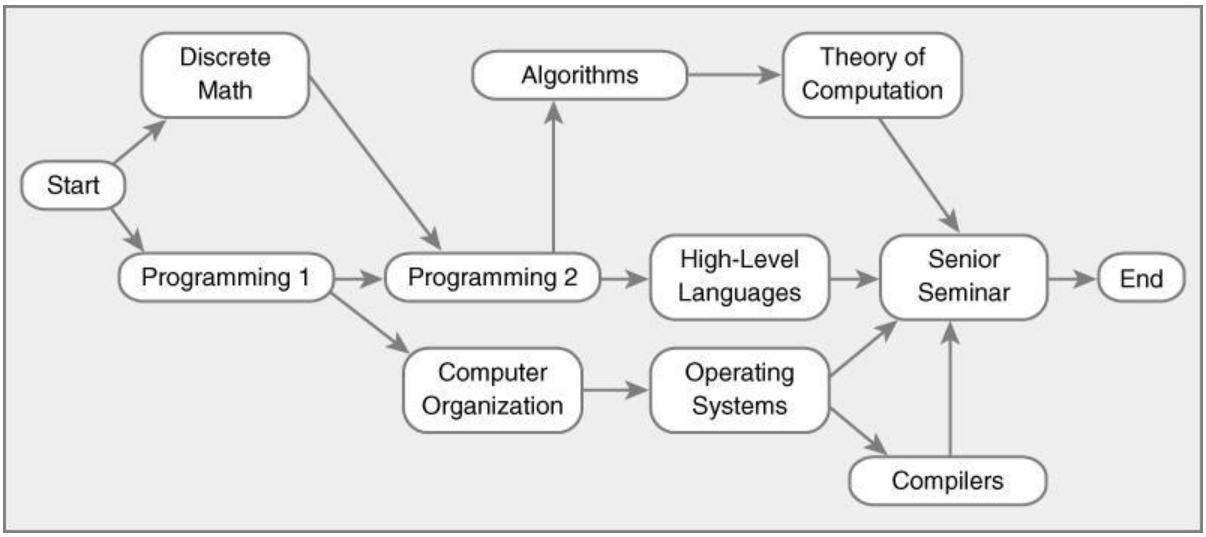
*To fill initial queue look for nodes w/o arrows pointing to them bc it means predCount = 0

Node	0	1	2	3	4	5	6	7	8	9
predCount	0	1	1	3	3	2	2	0	2	2

Step	Queue (Start of Step)	Node Removed	Neighbors Updated	topologicalOrder
1	[0, 7]	0	1, 5 (1 no longer has any arrow poinnting to it)	[0]
2	[7, 1]	7	4, 6, 9	[0, 7]
3	[1]	1	2,3,4, 5, 6	[0, 7, 1]
4	[2,5,6]	2	4	[0, 7, 1, 2]
5	[5,6, 4]	5	8	[0, 7, 1, 2, 5]
6	[6, 4]	6	8	[0, 7, 1, 2, 5, 6]
7	[4,8]	4	3	[0, 7, 1, 2,5,6,4]
8	[8, 3]	8	9	[0, 7, 1, 2, 5, 6, 4, 8]
9	[3. 9]	3	none	[0, 7, 1, 2, 5, 6, 4, 8, 3]
10	[9]	9	none	[0, 7, 1, 2,5,6,4,8,3,9]

Topological Order = [0,7,1,2,5,6,4,8,3,9]

10. List the nodes of the graph in a breadth first topological ordering.



Node	Start	Discrete Math	Programming 1	Programming 2	Algorithms	Computer Organization	Theory of Computation	High-Level Languages	Senior Seminar	Operating Systems	Compilers	End
predCount	0	1	1	2	1	1	1	1	4	1	1	1

Step	Queue (Start of Step)	Node Removed	Neighbors Updated	topologicalOrder
1	[Start]	**Start**	Discrete Math, Programming 1	[Start]
2	[Discrete Math, Programming 1]	**Discrete Math**	Programming 2	[Start, Discrete Math]
3	[Programming 1]	**Programming 1**	Programming 2, Computer Organization	[Start, Discrete Math, Programming 1]
4	[Programming 2, Computer Organization]	**Programming 2**	High-Level Languages, Algorithms	[Start, Discrete Math, Programming 1, Programming 2]
5	[Computer Organization, High-Level Languages, Algorithms]	**Computer Organization**	Operating Systems	[Start, Discrete Math, Programming 1, Programming 2, Computer Organization]
6	[High-Level Languages, Algorithms, Operating Systems]	**High-Level Languages**	Senior Seminar	[Start, Discrete Math, Programming 1, Programming 2, Computer Organization, High-Level Languages]
7	[Algorithms, Operating Systems]	**Algorithms**	Theory of Computation	[Start, Discrete, Pro1, Pro2, Comp Org High-Lvl Lang, Algorithms]
8	[Operating Systems, Theory of Computation]	**Operating Systems**	Compilers, Senior Seminar	[Start, Discrete, Pro1, Pro2, Comp Org High-Lvl Lang, Algorithms, Operating Systems]
9	[Theory of Computation, Compilers]	**Theory of Computation**	Senior Seminar	[Start, Discrete, Pro1, Pro2, Comp Org High-Lvl Lang, Algorithms, Operating Systems, Theory of Computation]
10	[Compilers]	**Compilers**	Senior Seminar	[Start, Discrete, Pro1, Pro2, Comp Org High-Lvl Lang, Algorithms, Operating Systems, Theory of Computation, Compilers]
11	[Senior Seminar]	**Senior Seminar**	End	[Start, Discrete, Pro1, Pro2, Comp Org High-Lvl Lang, Algorithms, Operating Systems, Theory of Computation, Compilers, Senior Seminar]
12	[End]	**End**	none	[Start, Discrete, Pro1, Pro2, Comp Org High-Lvl Lang, Algorithms, Operating Systems, Theory of Computation, Compilers, Senior Seminar, End]